Cluster and Cloud Computing – Lectures Week 3
HPC@UniMelb

Professor Richard O. Sinnott
Director, Melbourne eResearch Group
University of Melbourne
rsinnott@unimelb.edu.au

Outline

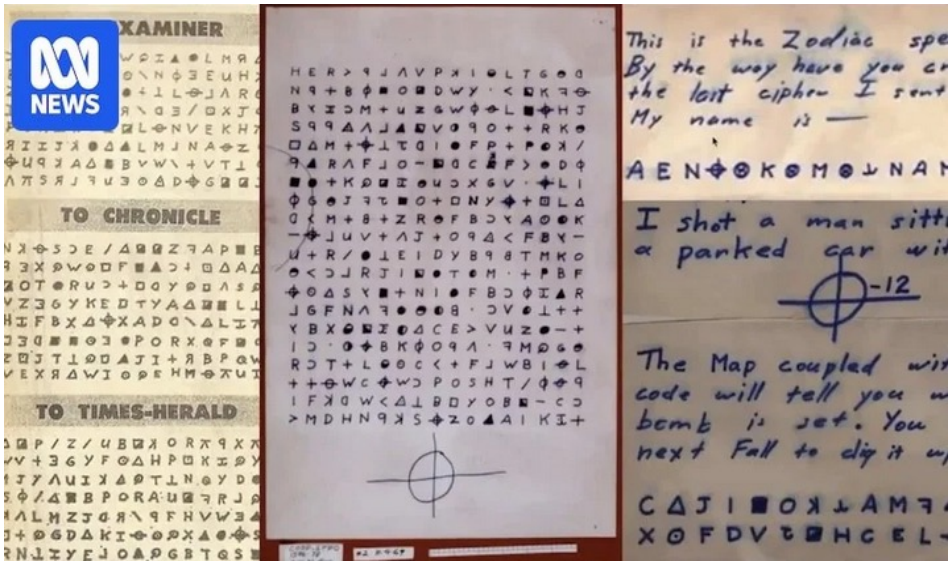
- This lecture includes:
  - Some background on supercomputing, high performance computing, parallel computing, research computing
  - An introduction to Spartan, University of Melbourne's HPC general purpose HPC system
  - Logging in, help, and environment modules
  - Job submission with Slurm workload manager; simple submissions, multicore, job arrays, job dependencies (job arrays), interactive jobs
  - Workshops will focus on MPI and mpi4py

Supercomputers

- As noted…
  - Data size and complexity is growing much faster than the capacity of personal computational devices to process that data
  - Various technologies to tackle this:
    - multi-processor systems, multi-core processors, shared and distributed memory parallel programming, general purpose graphics processing units, network improvements etc
  - All of these are typically incorporated in HPC systems at scale
  - Supercomputers underpin climate models, geophysics, biomolecular modelling, aerospace engineering, radio telescope data processing, particle physics, brain science, etc

Local Examples

- Crime solving
  - In 2019 a research team involving UniMelb / Spartan, broke the Zodiac cipher 360 which had eluded criminal and legal teams in the United States for over fifty years
  - See



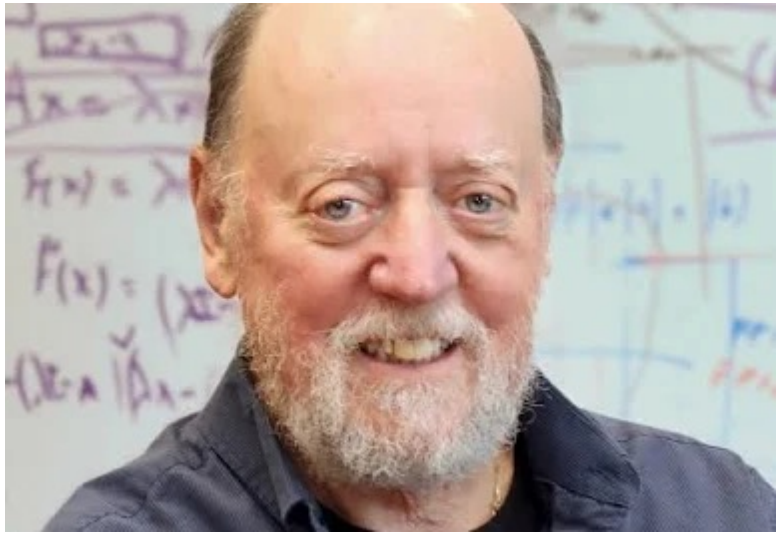
https://www.reddit.com/r/australia/comments/kbcmlh/zodiac_killer_code_cracked_by_australian/

Local Examples

- Drug development
  - Quantum simulation of biological systems
    - https://www.unimelb.edu.au/newsroom/news/2024/july/breakthrough-in-high-performance-computing-and-quantum-chemistry-revolutionises-drug-discovery
  - Gordon Bell Prize (Nobel Prize of HPC)
    - https://www.unimelb.edu.au/newsroom/news/2024/november/melbourne-researchers-win-the-nobel-of-high-performance-computing

What is a Supercomputer?

- "Supercomputer" is an arbitrary term.
  - any single computer system that has exceptional processing power for its time…
- One metric for measuring a supercomputer is the number of floating-point operations per second (FLOPS) such a system can carry out
  - LINPACK is a common benchmark
    - Solves dense n * n linear equations



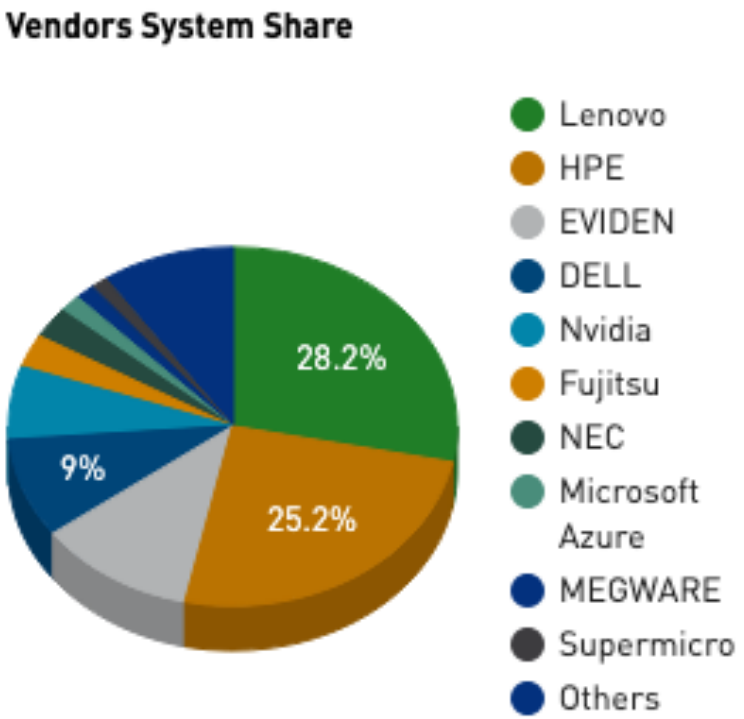
Fastest Supercomputers?

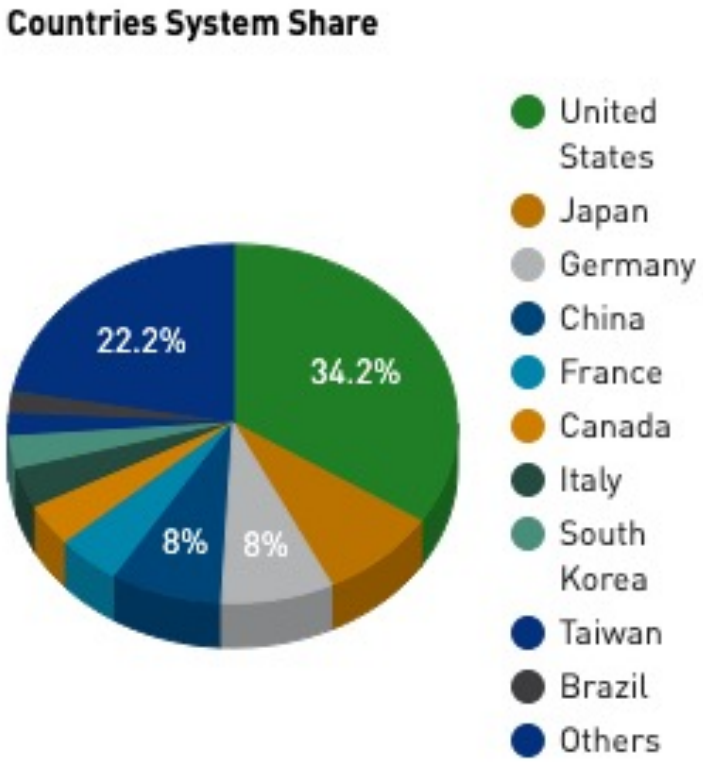
- Top500 - https://top500.org

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,340,000	1,809.00	2,821.10	29,685
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	1,000.00	1,226.28	15,794
5	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
6	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461
7	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
8	**Alps** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS, HPE Swiss National Supercomputing Centre (CSCS) Switzerland	2,121,600	434.90	574.84	7,124
9	**LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	531.51	7,107
10	**Leonardo** - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy	1,824,768	241.20	306.31	7,494

Fastest Supercomputers?

- Top500 - https://top500.org
  - Note geopolitical considerations too…





Greenest Supercomputers?

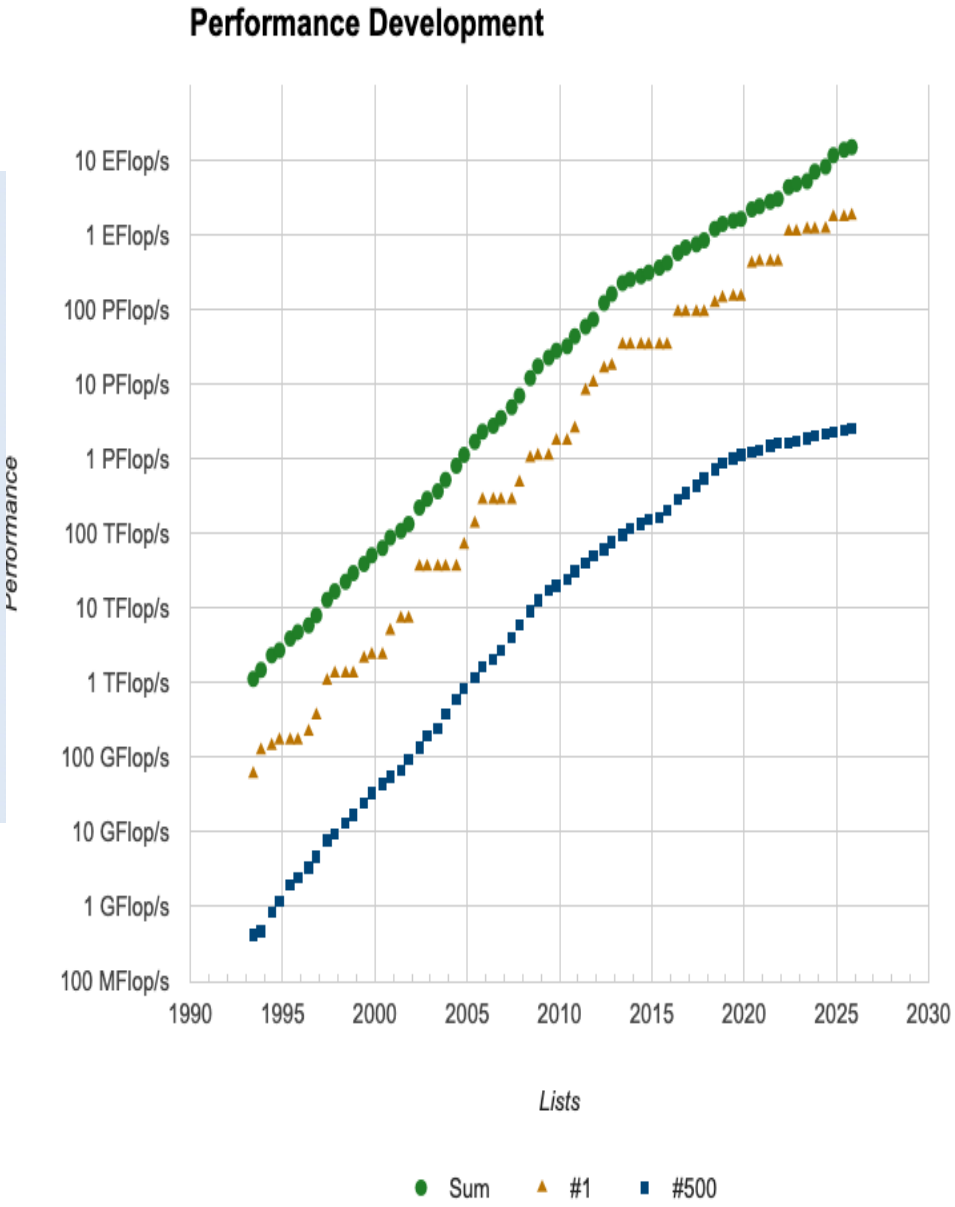
- Top500 - https://top500.org/lists/green500/

Rank	TOP500 Rank	System	Cores	Rmax (PFlop/s)	Power (kW)	Energy Efficiency (GFlops/watts)
1	420	**KAIROS** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN CALMIP / University of Toulouse - CNRS France	13,056	3.05	46	73.282
2	171	**ROMEO-2025** - BullSequana XH3000, Grace Hopper Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, Red Hat Enterprise Linux, EVIDEN ROMEO HPC Center - Champagne-Ardenne France	47,328	9.86	160	70.912
3	225	**Levante GPU extension** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN DKRZ - Deutsches Klimarechenzentrum Germany	35,904	6.75	110	69.426
4	213	**Isambard-AI phase 1** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE University of Bristol United Kingdom	34,272	7.42	117	68.835
5	286	**Otus (GPU only)** - ThinkSystem SD665-N V3, AMD EPYC 9655 96C 2.6GHz, NVIDIA H100 SXM5 80GB, Infiniband NDR, Rocky Linux 9.4, Lenovo Universitaet Paderborn - PC2 Germany	19,440	4.66		68.177
6	73	**Capella** - Lenovo ThinkSystem SD665-N V3, AMD EPYC 9334 32C 2.7GHz, Nvidia H100 SXM5 94Gb, Infiniband NDR200, AlmaLinux 9.4, MEGWARE TU Dresden, ZIH Germany	85,248	24.06	445	68.053
7	334	**SSC-24 Energy Module** - HPE Cray XD670, Xeon Gold 6430 32C 2.1GHz, NVIDIA H100 SXM5 80GB, Infiniband NDR400, RHEL 9.2, HPE Samsung Electronics South Korea	11,200	3.82	69	67.251
8	96	**Helios GPU** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cyfronet Poland	89,760	19.14	317	66.948
9	426	**AMD Ouranos** - BullSequana XH3000, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Infiniband NDR200, RedHat Enterprise Linux, EVIDEN Atos France	16,632	2.99	48	66.464
10	70	**Portage** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, RHEL 8.9, HPE Hewlett Packard Enterprise United States	129,024	24.50	370	66.277

Supercomputing Trajectory

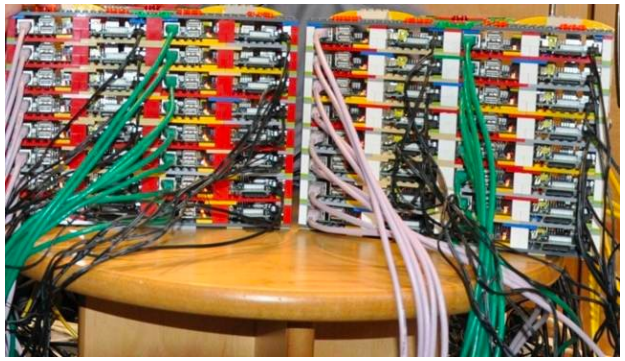
- Growing, growing, growing..

1993: 124.00 GFLOPS 1994: 170.00 GFLOPS 1995: 170.00 GFLOPS
1996: 368.20 GFLOPS 1997: 1.338 TFLOPS 1999: 2.3796 TFLOPS
2000: 7.226 TFLOPS 2001: 7.226 TFLOPS 2002: 35.860 TFLOPS
2003: 35.860 TFOPS 2004; 70.72 TFLOPS 2005: 280.6 TFLOPS
2006: 280.6 TFLOPS 2007: 478.2 TFLOPS 2008: 1.105 PFLOP
2009: 1.759 PFLOPS 2010: 2.566 PFLOPS 2011: 10.51 PFLOPS
2012: 17.59 PFLOPS 2013: 33.86 PFLOPS 2014: 33.86 PFLOPS
2015: 33.86 PFLOPS 2016: 93.01 PFLOPS 2017: 93.01 PFLOPS
2018: 143.00 PFLOPS 2019: 148.60 PFLOPS 2020: 442.01 PFLOPS
2021: 442.01 PFLOPS 2022: 1.102 EFLOPS 2023: 1.194 EFLOPS
2024: 1.742 EFLOPS



High Performance Computing (HPC)

- HPC is any computer system whose architecture provides *above average performance*
- Cluster computing is when two or more separate computers are used to offer a single resource
  - Improves performance and provides redundancy
  - Typically, a collection of smaller computers connected by a *high-speed* local network
    - Lack of single address space is the norm
  - Even a collection of Raspberry Pi's with Lego chassis is a cluster (University of Southampton, 2012)!



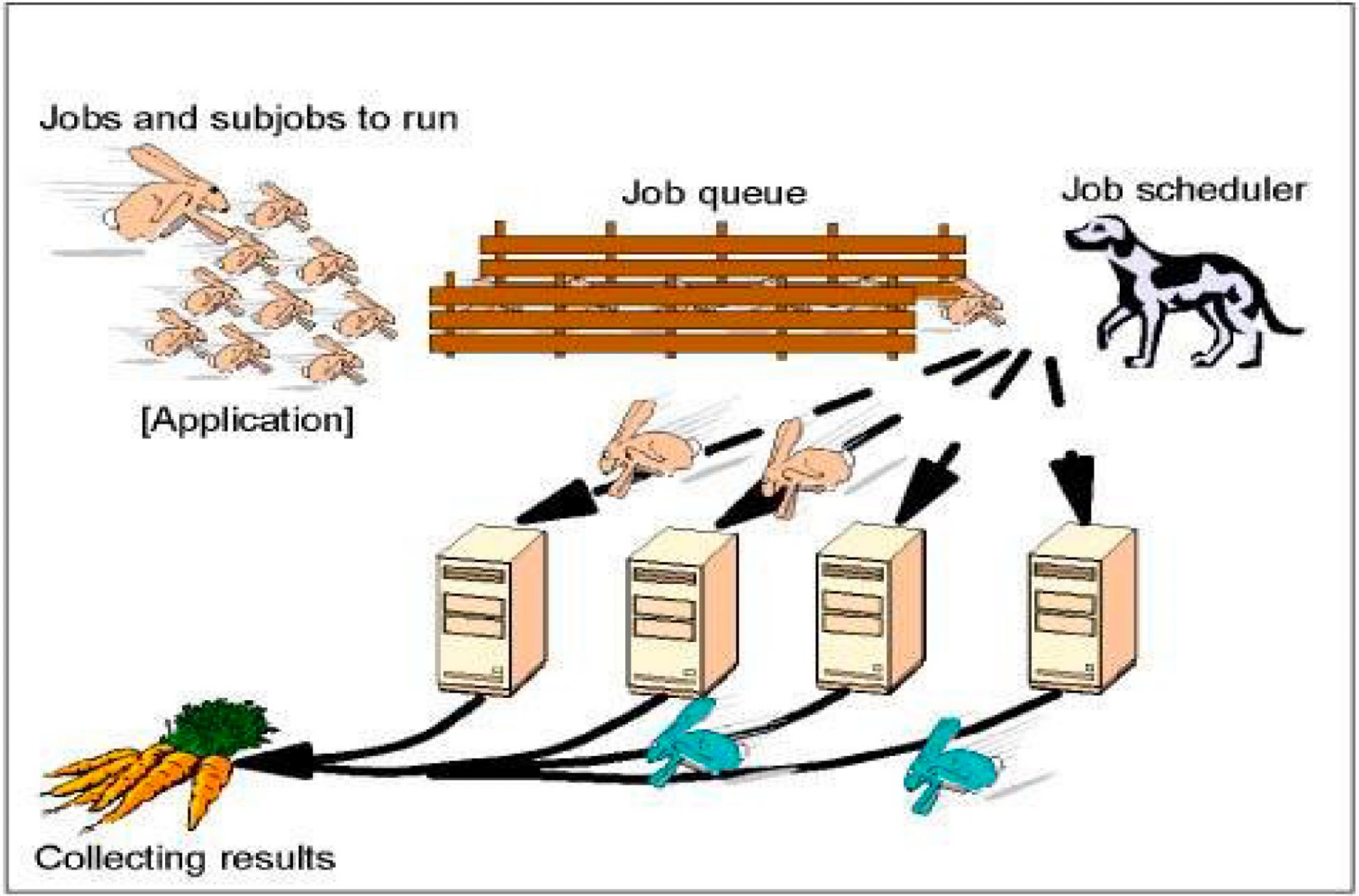
Parallel Programming

- With a cluster architecture, applications need to be parallelised to use the different computers
- Parallel computing refers to the submission of jobs or processes over multiple processors and splitting up the data or tasks between them
- Also introduces challenges - not just complexity of coding challenges:
  - Reproducibility challenges
    - Software versions, compilers and options, etc.
  - Running in parallel on shared system challenges
    - Benchmarking non-trivial
  - See: *The Ten-Year Reproducibility Challenge*
    - https://www.nature.com/articles/d41586-020-02462-7

Batch-based Systems

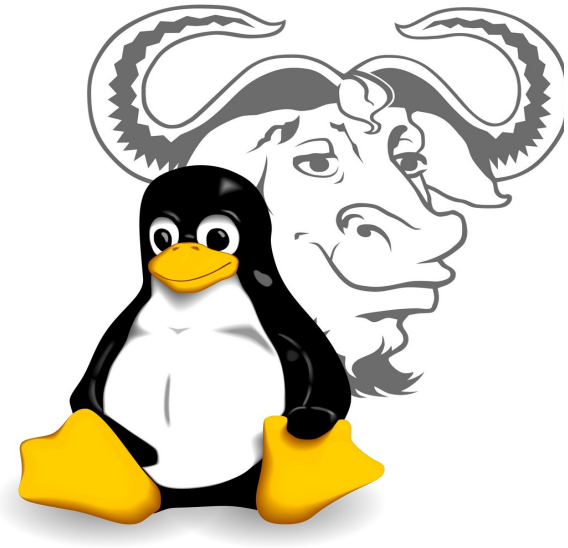
- Portable Batch System (PBS)
  - originally developed in 1990s (NASA)
  - Open source-version (OpenPBS) released in 1998
  - Released as Terascale Open-source Resource and QUEue Manager (TORQUE)
- PBS performs job scheduling
  - matches batch jobs resource requests to available resources (worker node compute resources)
- Simple Linux Utility for Resource Management (SLURM) enhanced version of PBS
  - SLURM used on SPARTAN
    - Local adaptations supported
      - e.g. site-specific queues, user projects for accounting

The “core” cluster idea…



Linux as the OS of Choice

- Since November 2017 every supercomputer on the Top 500 list has been based on Linux.
  - Linux scales and does so with stability and efficiency.
- Most scientific programs are designed to work with (GNU) Linux
- Linux and the vast majority of related applications are:
  - Free/open source - so financial savings
  - Community-wide improvement/optimization of systems
  - Can be compiled from source for the specific hardware used and optimised based on compiler flags
    - Necessary where every clock cycle is important



Introduction to SPARTAN

- Builds on history of HPC and Cloud resources over time at UniMelb
  - used/shared by many University researchers
- Naming evolution and capability
  - Alfred – King of England (2008)
    - just pure HPC with rack-mounted servers in data centre (more later)
  - Edward – King of Wessex (2011)
    - 2014+ included Cloud resources (more later)
  - *Aethelstan / Aelfweard*
    - not catchy enough!
  - SPARTAN – Greek warriors
    - Now includes extensive GPU resources
    - …everyone wants AI!!!

SPARTAN Offerings

Partition	Nodes	Cores per node	Memory per node (MB)	Processor	Peak Performance (DP TFlops)	Slurm node types	Extra notes
interactive	4	128	977000	Intel(R) Xeon(R) Gold 6448H	2.53	sr,avx512	Max wall clock limit of 2 days
long	2	72	710000	Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz	1.26	physg5,avx512	Max wall clock limit of 90 days
bigmem	9	72	2970000	Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz	5.07	physg5,avx512	Max wall clock limit of 21 days
	6	128	4060000	Intel(R) Xeon(R) Gold 6448H	5.07	sr,avx512	Max wall clock limit of 21 days
cascade	33	72	710000	Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz	46.89 (0.63 per node)	physg5,avx512	Max wall clock limit of 30 days
	2	72	1492048	Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz	46.89 (0.63 per node)	physg5,avx512	Max wall clock limit of 30 days
sapphire	78	128	976990	Intel(R) Xeon(R) Gold 6448H	143.75 (1.84 per node)	sr,avx512	Max wall clock limit of 30 days
	26	128	2001000	Intel(R) Xeon(R) Gold 6448H	143.75 (1.84 per node)	sr,avx512	Max wall clock limit of 30 days
gpu-a100	29	32	479910	Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz		avx512	4 80GB A100 Nvidia GPUs per node. Max wall clock limit of 7 days
gpu-a100-short	2	32	479910	Intel(R) Xeon(R) Gold 6326 CPU @ 2.90GHz		avx512	4 80GB A100 Nvidia GPUs per node. Max wall clock limit of 4 hrs
gpu-h100	16	64	950000	Intel(R) Xeon(R) Platinum 8462Y+ @ 2.80GHz		avx512	4 80GB H100 SXM5 Nvidia GPUs per node. Max wall clock limit of 7 days
gpu-l40s	10	64	950000	Intel(R) Xeon(R) Platinum 8562Y+ @ 2.80GHz		avx512	4 48GB L40S Nvidia GPUs per node. Max wall clock limit of 7 days
Total					199.5 (CPU) + 1358 (GPU)		

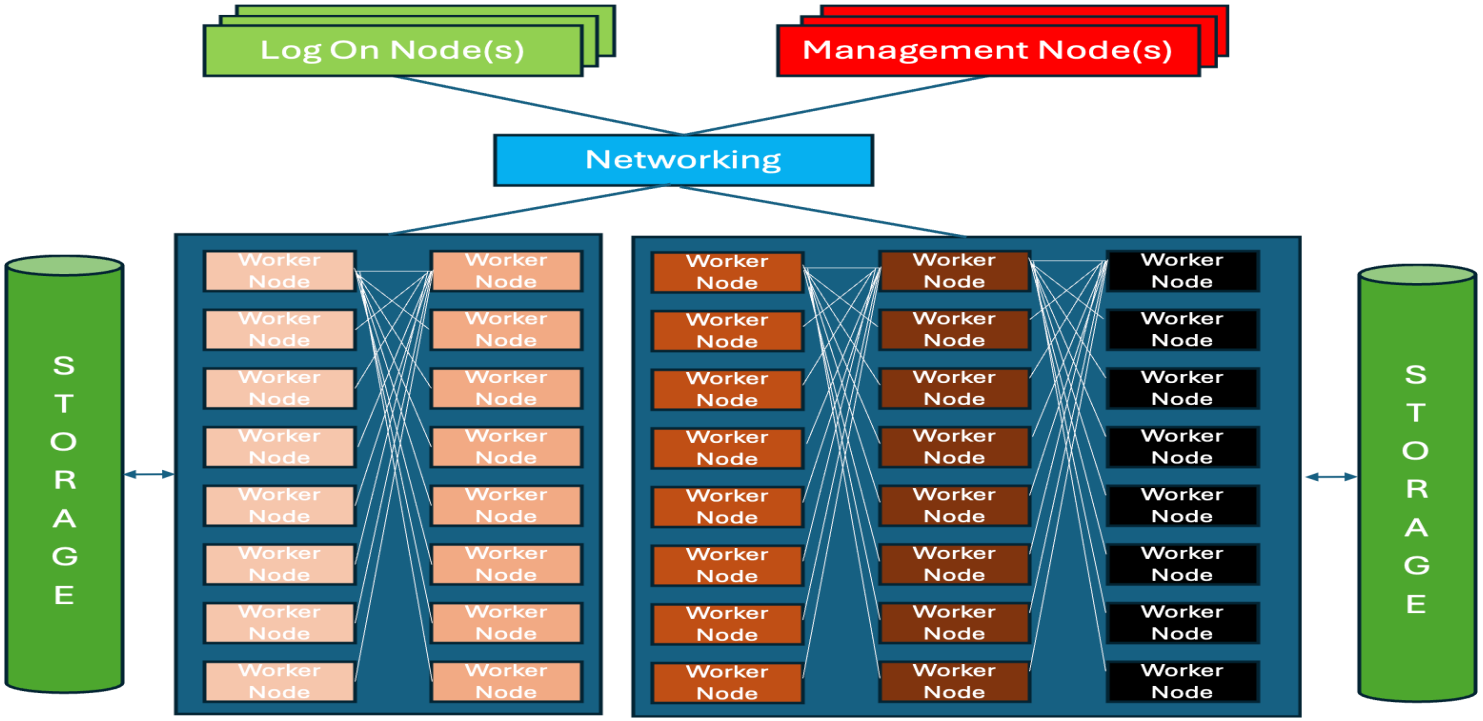
SPARTAN Offerings (Storage)

- General Parallel File System (*GPFS)
  - highly scalable, parallel file system
    * Now branded as IBM Storage Scale

Location	Capacity	Disk type
/data/gpfs	4.18PB	10K SAS
/data/scratch	761TB	NVMe

SPARTAN Architecture

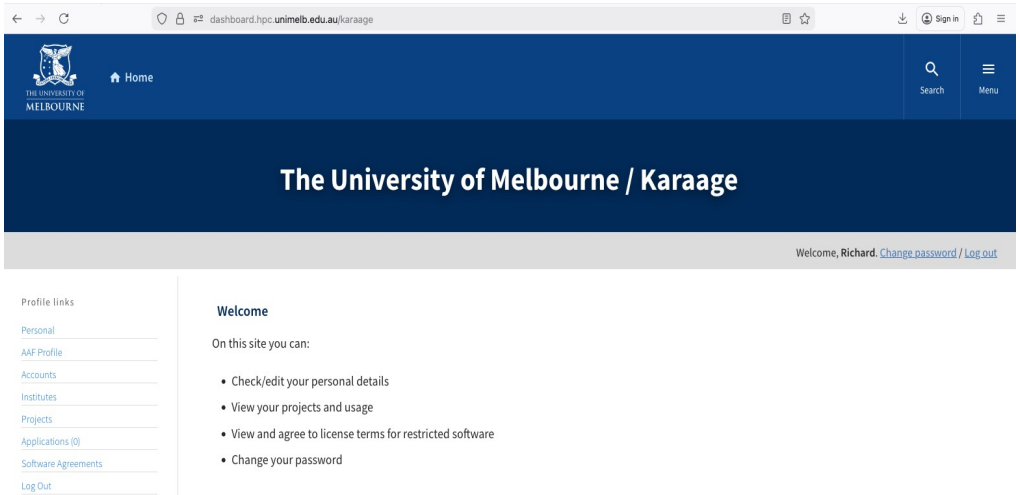
- Key components – typical HPC set up
  - **Management Node(s) -** for queueing system and job scheduler
  - **Logon Node(s) -** for user access to cluster
  - **Storage** - accessible (mounted) to all worker nodes
    - Some local storage on worker nodes also available
  - **Network switch -** for interconnecting everything
  - **Worker Nodes** - the workhorses for getting stuff done!



SPARTAN Recognition

SPARTAN Basics

- Spartan has its own authentication based on Security Assertion Markup Language (SAML)
  - https://dashboard.hpc.unimelb.edu.au/karaage
- Users on Spartan must belong to a project
  - Your project is called COMP90024
    - You apply on Karaage,
    - HPC admins will create your account (not me!)
      - They do this once or twice per day… not instantaneous!
    - You get added to SPARTAN (same Group)



SPARTAN Basics::Logging In

- After getting approved…
  - To log on to SPARTAN, you need a user account and password and a Secure Shell (ssh) client.
    - Linux distributions almost always include SSH as part of the default installation as does Mac OS 10.x.
    - For MS-Windows users, the free PuTTY client or MobaXterm is recommended. To transfer files to/from SPARTAN use scp, WinSCP, Filezilla, or rsync.
  - To log in type:
    - *ssh your-username@spartan.hpc.unimelb.edu.au*
      - *Enter the password you set up with karaage*
  - For help go to:
    - http://dashboard.hpc.unimelb.edu.au or type *man spartan or email*: hpc-support@unimelb.edu.au

SPARTAN Basics::Linux Environment

- SPARTAN is a Linux-based system
  - Basic introduction to Linux covered in workshop
- Linux uses a command line
  - Can get by in this assignment with knowledge of 20-30 commands
    - (There is a LOT more to it than this!!!)
- There is NO need to install software
  - Never use SUDO (*superuser do*) to try to elevate your privileges to install software
    - It will get flagged by the HPC admins immediately
  - All of the software you need is installed already
  - If you REALLY need to add something, then you can use a *virtualenv* (see workshop)
    - Not essential though…

SPARTAN Basics::Modules

- There are many (many!) software systems installed on SPARTAN
  - All built/optimized/deployed by the HPC admins to work on the SPARTAN hardware
    - They do this, no-one else!
- These are available through **MODULES**
  - provide for dynamic modification of the user's environment (*paths*)
  - Each module contains configuration information for the user session, e.g. location of application executables loaded/available at that time
  - Modulefiles used by many users allowing multiple installations of the same application but with different versions and compilation options

SPARTAN Basics::Modules

- There are several commands associated with modules
  - **module help**
    - provides a list of the switches, subcommands, and subcommand arguments that are available through the environment modules package.
  - **module avail**
    - lists all the modules which are available to be loaded
  - **module whatis <modulefile>**
    - provides a description of the module listed
  - **module display <modulefile>**
    - show what a given modulefile will do to your environment, such as what will be added to the PATH, MANPATH, etc.
  - **module list**
    - List currently loaded modules

SPARTAN Basics::Modules…ctd

- **module load <modulefile>**
  - This adds one or more modulefiles to the user's current environment (some modulefiles load other modulefiles
- **module unload <modulefile>**
  - This removes any listed modules from the user's current environment
- **module switch <modulefile1> <modulefile2>**
  - Unloads one modulefile (modulefile1) and loads another (modulefile2)
- **module purge**
  - This removes all modules from the user's environment

- Also
  - **module spider**
    - searches for all possible modules and not just those in the existing module path

SPARTAN Software Again

- What if you really need software that does not exist as a module? You shouldn't, but…
  - Ask the HPC admins nicely
    - They may well say no!
  - Use a **virtual environment (venv)**
    - **See workshop for how to do this**

SPARTAN Basics::Job submission/scheduling

- How to run a job
  - Create a script (slurm file)
  - Submit the script to the job scheduler
    - sbatch <*jobscript*> [params]

- Essential…
  - DO NOT RUN BIG JOBS ON THE LOG-IN NODE
  - If you run for example
    - *Python mycode.py bluesky-large.ndjson* (130Gb file!)
    - You will be running on the Log In node and have a good chance of seriously impacting/bringing down the system
    - Always use sbatch and put the job in a script

SPARTAN Basics::Job submission/scheduling

- What is in a SLURM script?
  - See https://slurm.schedmd.com/quickstart.html for broader description of SLURM

    - Start the shell (#!/bin/sh)
    - What queue (partition do you want to use)?
      - Suggest to use sapphire
      - Many queues are inaccessible (e.g. just for physics etc)
    - How long will the job run for (walltime)?
      - Default is 15 minutes
      - Scheduler will kill all jobs if walltime reached
    - How many nodes/cores/tasks do you want/need?
    - What modules do you need to load?
    - How do you compile/run your code?

SPARTAN Basics::Job submission/scheduling

- Example SLURM script and most basic Python code

testPy.sh

```
#!/bin/sh
#SBATCH --job-name=testPy
#SBATCH --time=00:10:00 #(hrs:min:sec)
#SBATCH --nodes=1 #(run on a single server)
#SBATCH --ntasks=4 #(run 4 tasks)
#SBATCH --cpus-per-task=1 #(CPU cores per task)
#Load required modules
module purge
module load GCC/11.3.0
module load OpenMPI/4.1.4
module load mpi4py/3.1.4
mpirun python testPy.py
```

????

mpirun?

Sbatch testPy.sh

testPy.py

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
print ("hello from process",
        rank, "of", size)
```

MPI_Init :initiate MPI computation
MPI_Finalize : terminate computation

???

SPARTAN Basics::Job submission/scheduling

- What about…?

testPy2.sh

```
#!/bin/sh
#SBATCH --job-name=testPy
#SBATCH --time=00:10:00 #(hrs:min:sec)
#SBATCH --nodes=1 #(run on a single server)
#SBATCH --ntasks=4 #(run 4 tasks)
#SBATCH --cpus-per-task=1 #(CPU cores/task)
#Load required modules
module purge
module load GCC/11.3.0
module load OpenMPI/4.1.4
module load mpi4py/3.1.4
python testPy.py
```

- What about…?

testPy3.sh

```
#!/bin/sh
#SBATCH --job-name=testPy
#SBATCH --time=00:10:00 #(hrs:min:sec)
#SBATCH --nodes=1 #(run on a single server)
#SBATCH --ntasks=4 #(run 4 tasks)
#SBATCH --cpus-per-task=1 #(CPU cores/task)
#Load required modules
module purge
module load GCC/11.3.0
module load OpenMPI/4.1.4
module load mpi4py/3.1.4
mpirun python testPy.py
mpiexec python testPy.py
srun python testPy.py
```

SPARTAN Basics::Job management

- **System state and queues?**

  sinfo –s //check partition status

  squeue | less //check the queues

  showq -p <partition> | less //check specific partition

- **Job status?**
  - squeue –j <jobid> //check specific job status
  - scancel <jobid> //cancel specific job

- **Outputs?**
  - Slurm-NNNNNNNN.out

    //output to the specific directory where job

    //launched unless otherwise directed

Other Scenarios (not needed for ass1)

- What about more complex scenarios?
  - Sending and managing many jobs
- Array Jobs
  - e.g. apply the same task across multiple datasets
  - e.g. submit 10 batch jobs with *myapp* running against datasets dataset1.csv,... dataset10.csv

```
#SBATCH --array=1-10
myapp ${SLURM_ARRAY_TASK_ID}.csv
```

- What if dependencies between jobs?
  - Output of one job is input to another?
    - dependency directives such as`after`, `afterok`, `afternotok`, `before`, `beforeok`, `beforenotok`
    - For example

```
#SBATCH --dependency=afterok:myfirstjobid mysecondjob
```

- See SPARTAN /apps/examples/Python/Array for examples

Other scenarios (not needed for ass1)

- Batch-based only, i.e. submit and results later?
  - What if want to do some reasonably compute heavy tasks but don’t want to kill the login node?

- **Interactive jobs**
  - Put user onto actual node(s)

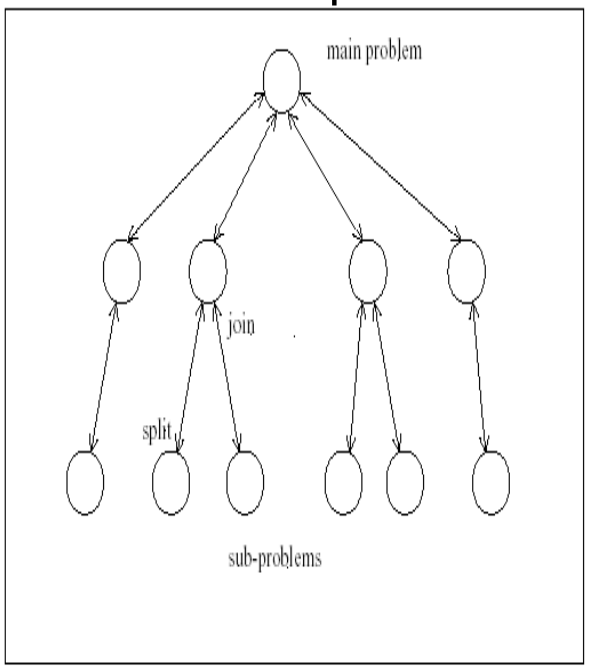
```
$ sinteractive --nodes=1 --ntasks-per-node=1

$ sinteractive --nodes=2 --ntasks-per-node=1
```

Shared Memory Jobs

- openMPI – ideal for message passing
- openMP – ideal for multithreaded jobs
  - Remember week2

Task Decomposition



Shared Memory Jobs…ctd

- Consider hello3omp.c

```
#include <stdio.h>
#include  "omp.h"
int main(void)
{ char greetingsen[] = "Hello world!";
  char greetingsde[] = "Hallo Welt!";
  char greetingsfr[] = "Bonjour le monde!";
  int a,b,c;
  #pragma omp parallel sections
  { #pragma omp section
      { for ( a = 0; a < 3; a = a + 1 )
          {printf ("id = %d, %s\n",
              omp_get_thread_num(), greetingsen);}}
  #pragma omp section
      {for ( b = 0; b < 3; b = b + 1 )
          {printf ("id = %d, %s\n",
             omp_get_thread_num(), greetingsde);}}
  #pragma omp section
      {for ( c = 0; c < 3; c = c + 1 )
          {printf ("id = %d, %s\n",
             omp_get_thread_num(), greetingsfr);}}}
return 0;
}
```

- Compile

  gcc –fopenmp hello3omp hello3omp.c

- Run

  ./hello3omp

  vs

- Sbatch Hello3omp.slurm

```
#!/bin/bash
#SBATCH --time=0:01:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
module purge
module load GCC/13.3.0
module load OpenMPI/5.0.3
export OMP_NUM_THREADS=8
gcc -fopenmp -o hello3omp hello3omp.c
./hello3omp
```

More examples

- See SPARTAN
  - /apps/examples/ {all kinds of things here!!!!)
  - /apps/examples/OpenMP {openMP examples here}
  - /apps/examples/OpenMPI {openMPI examples here}
  - /apps/examples/Python {Python examples here}
  - /apps/examples/Python/MPI {MPI Python examples here}
  - NOTE!!!! Scripts will need tweaking (based on older modules!)

- Note you can copy these examples to your home directory and play around with them but you can't run them in directory /apps/examples etc

```
$mkdir myexamples
$cd myexamples
$cp -r /apps/examples/Python/MPI/ .
```

Questions?